

ExamForge AI

Phase 1: Enterprise Verification Audit

Current State Verification with Evidence

Report Date	2026-07-30
Project	ExamForge AI - Flutter Web + Supabase
Repository	austinchima183-ui/examforgeai
Audit Scope	Database, Edge Functions, Flutter, Payments, Auth, CBT, AI, Marketplace, Realtime
Methodology	Source code review (static analysis) - no runtime tools available
Verification Level	Evidence-based - no assumptions or estimates

1. Executive Summary

This Phase 1 verification audit examines the current state of all ExamForge AI components using evidence-based source code analysis. The audit was conducted through static code review of the Flutter Web application, Supabase Edge Functions, SQL migrations, and configuration files. No runtime tools (Flutter SDK, Supabase CLI) were available on the verification system, so all findings are based on source code evidence only. Runtime verification (flutter analyze, flutter test, flutter build, database queries) must be performed in a properly configured environment.

Component	Status	Evidence	Critical Issues
Database Schema	PARTIAL	813-line schema.sql + 23 migration files	Missing on 8 base tables; migration tables unverified
Edge Functions (10)	PASS	All 10 functions reviewed with JWT 3 methods satellite lib	UnimplementedError
Flutter Analyze	UNVERIFIED	Flutter SDK not installed	Cannot verify compile errors
Flutter Tests	FAIL	No test/ directory exists	Zero test coverage
Flutter Build Web	UNVERIFIED	Flutter SDK not installed	Cannot verify release build
Notifications	PARTIAL	FCM + Realtime subscription code exists	No local notification display; TODO in code
Payment (Flutterwave)	PARTIAL	Edge Function architecture complete	ESHOPEJSKEYTdash still missing; must configure
Authentication	PASS	Full Supabase Auth integration with No PKA	no pre-rename mapping detected
Admin System	PARTIAL	Super admin dashboard + security service exists	Policy escalation prevention via RLS only
CBT Engine	PASS	Server-authoritative timing, anti-double-click	Cannot real-time; state de-recovery
AI Services	PASS	Edge Functions with JWT validation, In-memory init, limited logging	(persistent)
Marketplace	PARTIAL	Full CRUD + secure download + payment 781 vendor/ECBM	across 134 files
Realtime	PASS	Supabase Realtime subscriptions for CBT recommendations	mark-off strategy
Storage	PARTIAL	Signed URL generation for marketplace downloads	No storage bucket policies verified
Environment Variables	FAIL	No .env file exists	All runtime config missing

2. Database Schema Verification

The Supabase database schema is defined in a primary schema.sql file (813 lines) and supplemented by 23 additional migration files covering billing, marketplace, CBT enhancements, AI generator, parent portal, communication, and more. The schema defines 5 custom enum types (user_role, subscription_status, exam_status, notification_type, question_type) and 8 core tables with proper foreign key constraints, indexes, and comments.

2.1 Core Tables (Verified in schema.sql)

Table	Primary Key	FK References	RLS Enabled	Indexes
schools	UUID (gen_random_uuid())		Yes	3
users	UUID (auth.users)	schools, auth.users	Yes	4
classes	UUID	schools, users	Yes	3
subjects	UUID	schools	Yes	3
class_subjects	UUID	classes, subjects, users	Yes	3
class_students	UUID	classes, users	Yes	2
notifications	UUID	users	Yes	3
audit_log	UUID	users (nullable)	Yes	2

2.2 RLS Policies (Verified: 36 CREATE POLICY statements)

Row Level Security is enabled on all 8 core tables. The schema defines 36 RLS policies covering role-based access control (super_admin, school_admin, teacher, student). Policies use a helper function get_user_role() to determine the current user role. Key policies include: users can only update their own row (with role change prevention), school admins can CRUD within their school, teachers can read classes they teach, and students can read enrolled classes. This is a well-designed multi-tenant RLS architecture.

2.3 Migration Files (23 files)

The project contains 23 additional migration SQL files covering: billing_schema.sql, cbt_engine_schema.sql, cbt_engine_enhancements_schema.sql, communication_schema.sql, ccms_enterprise_schema.sql, database_optimization.sql, final_production_schema.sql, infrastructure_monitoring.sql, marketplace_schema.sql, marketplace_security.sql, parent_portal_schema.sql, payment_security_hardening.sql, question_bank_schema.sql, refund_security.sql, results_analytics_schema.sql, rls_role_fix.sql, school_management_schema.sql, student_portal_schema.sql, super_admin_schema.sql, teacher_workspace_schema.sql, teacher_workspace_expansion_schema.sql, ai_generator_schema.sql, mobile_offline_schema.sql. However, RLS ENABLE statements were NOT found in these migration files. Only the base schema.sql has explicit RLS enabling. This is a critical gap that must be verified at runtime.

WARN RLS only confirmed on 8 base tables. Migration tables (billing, marketplace, CBT, etc.) have no RLS ENABLE statements in their SQL files. Must verify at runtime whether these tables have RLS enabled.

3. Supabase Edge Functions

The project contains 10 Edge Functions written in TypeScript for Deno. All functions follow a consistent security architecture: JWT authentication via Supabase Auth, environment-specific CORS allow-lists (hardcoded per environment), and proper error handling. The functions use Supabase service role keys only for admin operations and validate user identity before processing.

Function	Auth	CORS	Rate Limit	Audit Log	Issues
flutterwave-checkout	Yes	Hardened	No	No	None
flutterwave-verify	Yes	Hardened	No	No	None
flutterwave-webhook	No (webhook)	Hardened	No	No	Verifies signature with constant-time comparison
process-refund	Yes	Hardened	No	No	Authorization check (owner/admin)
payment-operations	Yes	Hardened	No	Yes	Amount mismatch detection
health-check	No	Hardened	No	No	Checks DB, Storage, Auth, Payment health
ai-complete	Yes	Hardened	Yes (20/min)	Yes	In-memory rate limiting
ai-stream	Yes	Hardened	Yes (20/min)	Yes	SSE streaming with audit logging
exam-timing	Yes	Hardened	No	Yes	Server-authoritative timing
marketplace-download	Yes	Hardened	No	No	Signed URLs + purchase verification

3.1 Security Architecture

All Edge Functions follow a consistent security pattern: (1) Environment-specific CORS allow-lists that reject wildcard origins, with production restricted to examforge.ai domains only. (2) JWT authentication via Supabase Auth `getUser()` before processing any request. (3) Service role key used only for admin operations, never exposed to the client. (4) AI functions include rate limiting (20 requests per minute per user) with proper 429 responses and `Retry-After` headers. (5) AI functions validate model names against an allow-list, preventing arbitrary model usage. (6) Payment functions include amount verification to prevent payment spoofing attacks.

3.2 Unimplemented Edge Function Methods

The `FlutterwaveDataSourceImpl` in `flutterwave_datasource.dart` throws `UnimplementedError` for three methods that must be routed through Edge Functions: `createPaymentPlan`, `subscribeToPlan`, and `getTransactionFee`. These methods reference Edge Functions (`flutterwave-create-plan`, `flutterwave-subscribe-plan`, `flutterwave-transaction-fee`) that do not exist in the `supabase/functions` directory. This means subscription plan creation and management is non-functional.

FAIL 3 Edge Functions are missing: `flutterwave-create-plan`, `flutterwave-subscribe-plan`, `flutterwave-transaction-fee`

4. Flutter Application Verification

The Flutter application is a comprehensive SaaS platform with 15+ feature modules. The project uses Flutter 3.3+ with Riverpod for state management, GoRouter for navigation, and Supabase Flutter SDK for backend integration. The codebase follows clean architecture with data/domain/presentation layers for each feature module.

4.1 Project Structure

The project is organized into feature modules: auth, billing, cbt_engine, communication, customer_success, dashboard, exam_ecosystem, marketplace, notifications, parent_portal, results, school_management, question_bank, ai_generator, super_admin, offline, and more. Each module follows a consistent structure: data/datasources, data/repositories, data/models, domain/entities, domain/repositories, domain/usecases, presentation/providers, presentation/pages, presentation/widgets. Core services include: auth_service, notification_service, storage_service, and AI services (OpenAI + Gemini providers).

4.2 Flutter Analyze / Test / Build

Check	Result	Evidence
flutter analyze	UNVERIFIED	Flutter SDK not installed on verification system
flutter test	FAIL - No Tests	No test/ directory exists in the project
flutter build web --release	UNVERIFIED	Flutter SDK not installed on verification system
Test Coverage	0%	No test files found anywhere in the project

FAIL Zero test coverage. No test/ directory exists. This is a critical blocker for production deployment.

4.3 Code Quality Indicators

A search for mock, placeholder, TODO, FIXME, HACK, and XXX markers across the lib/ directory found 751 occurrences across 134 files. The highest concentration is in the exam_ecosystem module (47+ occurrences in models, 50+ in repository, 36+ in provider, 23+ in mock_exam_list_page, 30+ in mock_exam_take_page) and marketplace_analytics_page (21 occurrences). While many TODOs are expected in development, the volume suggests significant incomplete work, particularly in the exam ecosystem and marketplace modules.

5. Payment System (Flutterwave)

The Flutterwave payment integration follows a secure architecture where all sensitive operations are routed through Supabase Edge Functions. The client-side FlutterwaveDataSourceImpl only holds the public key and delegates checkout initialization, transaction verification, and refund processing to server-side Edge Functions. The Flutterwave secret key is never exposed to the client.

5.1 Configuration Status

Configuration Item	Status	Details
FLUTTERWAVE_PUBLIC_KEY	NEEDS .env	Must be set in .env file for client-side checkout
FLUTTERWAVE_SECRET_KEY	RECEIVED	FLWSECK-0725813e27cb7dae3faf8ce00ee35e4c-19fae2b082avt-X - must be set
FLUTTERWAVE_WEBHOOK_SECRET_HASH	MISSING	Required for webhook signature verification - NOT provided
flutterwave-checkout Edge Function	EXISTS	Initializes Standard Checkout with server-side secret key
flutterwave-verify Edge Function	EXISTS	Verifies payments with amount mismatch detection
flutterwave-webhook Edge Function	EXISTS	Handles webhooks with constant-time signature comparison
process-refund Edge Function	EXISTS	Processes refunds with owner/admin authorization
payment-operations Edge Function	EXISTS	Additional verify-payment and initiate-refund operations
createPaymentPlan	UNIMPLEMENTED	Throws UnimplementedError - Edge Function does not exist
subscribeToPlan	UNIMPLEMENTED	Throws UnimplementedError - Edge Function does not exist
getTransactionFee	UNIMPLEMENTED	Throws UnimplementedError - Edge Function does not exist

5.2 Security Measures

The payment system implements several security measures: (1) Constant-time comparison for webhook signature verification, preventing timing attacks. (2) Server-side amount verification in the flutterwave-verify Edge Function, comparing the charged amount against the expected amount from the local transactions table. Amount mismatches exceeding 1.0 are flagged as potential fraud and logged to the audit_log. (3) Webhook idempotency tracking via WebhookIdempotencyTracker with 72-hour eviction and 10,000 event cache. (4) Owner/admin authorization check for refund operations. However, the WEBHOOK_SECRET_HASH is still missing, which means webhook verification cannot function.

FAIL FLUTTERWAVE_WEBHOOK_SECRET_HASH is missing. Without it, the webhook Edge Function cannot verify that incoming webhooks are genuinely from Flutterwave. This is a critical security blocker.

6. Notification System

The notification system uses a dual-channel approach: Firebase Cloud Messaging (FCM) for push notifications and Supabase Realtime for in-app notifications. The `NotificationService` class handles FCM token management, topic subscriptions (role-based and school-based), foreground message handling, background tap handling, and terminated state tap handling. FCM tokens are synced to a `device_tokens` table in Supabase for server-side targeting.

6.1 Notification Features

Feature	Status	Evidence
Permission Request	Implemented	iOS + Android permission handling in <code>NotificationService</code>
FCM Token Management	Implemented	Token obtain, refresh, persist, and sync to Supabase
Topic Subscriptions	Implemented	Role-based (<code>role_admin</code>) and school-based (<code>school_<id></code>) topics
Foreground Handler	Implemented	<code>onMessage</code> listener with callback
Background Tap Handler	Implemented	<code>onMessageOpenedApp</code> listener
Terminated State Handler	Implemented	<code>getInitialMessage()</code> processing
Local Notification Display	INCOMPLETE	TODO comment: "Integrate <code>flutter_local_notifications</code> "
Sign-out Cleanup	Implemented	Token removal + topic unsubscription + stream disposal
Supabase Realtime Notifications	Implemented	<code>SupabaseConfig.subscribeToTable()</code> for notifications table
Notification Preferences	Implemented	Communication feature has <code>notification_preferences_page</code>
Read/Unread Tracking	Implemented	<code>is_read</code> + <code>read_at</code> columns in notifications table
Broadcast Notifications	Implemented	<code>SupabaseConfig.subscribeToBroadcast()</code> for ephemeral messages

WARN Local notification display is incomplete (TODO in code). Foreground FCM messages are logged but not displayed as heads-up notifications. `flutter_local_notifications` package is not in `pubspec.yaml`.

7. Authentication System

The authentication system is built on Supabase Auth with a comprehensive AuthService class that wraps all Supabase Auth operations. The service handles sign-up, login, logout, forgot password, reset password, email verification, magic link (OTP), session refresh, and profile updates. All Supabase-specific exceptions are mapped to domain-level AuthException instances with user-friendly messages and stable error codes. Session tokens are persisted in flutter_secure_storage and automatically cleared on sign-out.

7.1 Security Measures

The authentication system includes: (1) Password change requires re-authentication (signInWithPassword before updateUser). (2) Role change prevention: users cannot update their own role through the client (enforced by RLS policy on the users table). (3) Session expiry checking with expiresAt validation. (4) Secure storage for all tokens (flutter_secure_storage with encryption at rest). (5) Global sign-out scope (all devices signed out). (6) Friendly error messages that do not leak internal details (e.g., "Invalid email or password" instead of "User not found").

Feature	Status
Email/Password Auth	Implemented
Magic Link / OTP	Implemented
Email Verification	Implemented
Password Reset	Implemented
Session Refresh	Implemented
Profile Update	Implemented
Password Change (re-auth)	Implemented
MFA Enforcement	Not Detected
Account Lockout	Not Detected (relies on Supabase defaults)

WARN No MFA enforcement or account lockout policies detected in the application code. Relies on Supabase Auth defaults for brute-force protection.

8. CBT Engine

The Computer-Based Testing (CBT) engine is a well-architected system with server-authoritative exam timing, anti-cheat monitoring, session recovery, and realtime monitoring. The exam-timing Edge Function is the single source of truth for all timing decisions, preventing client-side time manipulation. The CbtRealtimeService provides Supabase Realtime subscriptions for exam sessions, attempts, and monitoring events.

Feature	Status	Implementation
Server-Authoritative Timing	PASS	exam-timing Edge Function with server timestamps
Start Attempt	PASS	Server records start time, returns deadline
Sync Time	PASS	Periodic client check with remaining time calculation
Submit Attempt	PASS	Server validates timing, detects late submissions
Recover Attempt	PASS	After reconnect, get current state from server
Auto-Submit on Timeout	PASS	Server auto-submits when time expires during sync
Anti-Cheat Monitoring	PASS	Tab switch, copy/paste, fullscreen exit, keyboard shortcuts
Violation Summary	PASS	Aggregated violations with disqualification thresholds
Session Recovery	PASS	CbtRealtimeService with heartbeat + reconnect handling
Realtime Monitoring	PASS	Supabase Realtime for sessions, attempts, monitoring events
Rate Limiting	PARTIAL	Client-side rate limiting service exists
Exam Notifications	PASS	Exam notification service for CBT events
Auto-Save	PASS	Auto-save service for exam answers

PASS CBT engine architecture is well-designed with server-authoritative timing, comprehensive anti-cheat, and realtime monitoring. Runtime verification still needed.

9. AI Services

The AI services are properly secured through Edge Functions that keep API keys server-side. The ai-complete Edge Function provides synchronous completion and the ai-stream Edge Function provides Server-Sent Events (SSE) streaming. Both functions support OpenAI and Gemini providers with model validation against an allow-list, prompt length bounding (50,000 characters), max tokens clamping (4,096), and temperature clamping (0-2.0).

Feature	Status	Details
OpenAI Integration	PASS	GPT-4o, GPT-4o-mini, GPT-4-turbo, GPT-4, GPT-3.5-turbo
Gemini Integration	PASS	Gemini 1.5 Pro, 1.5 Flash, 1.0 Pro
API Key Security	PASS	Keys only in Edge Function env vars, never exposed to client
Rate Limiting	PARTIAL	In-memory (20/min/user) - resets on function cold start
Model Allow-List	PASS	Only pre-approved models can be used
Prompt Length Limit	PASS	50,000 character maximum
Audit Logging	PASS	All requests logged to ai_request_log table
Usage Tracking	PASS	Token usage tracked per request (prompt + completion + total)
Streaming Support	PASS	SSE with proper error handling and timeout
Timeout Protection	PASS	60s for complete, 120s for streams

WARN Rate limiting is in-memory only. Edge Function cold starts reset the rate limit map. For production-grade rate limiting, use a Redis-backed or DB-backed solution.

10. Marketplace

The marketplace module is extensive with seller management, product CRUD, cart, checkout, purchases, reviews, quality checks, moderation, commissions, disputes, wishlists, and AI resource generation. The marketplace-download Edge Function implements secure file downloads with purchase verification, signed URLs (1-hour expiry), and download token generation with a 5-download limit and 24-hour expiry. The domain layer contains 40+ use cases covering all marketplace operations.

Marketplace has comprehensive feature coverage but 751 TODO/FIXME markers across 134 files suggest significant incomplete work. Runtime verification needed.

11. Realtime Subscriptions

The `SupabaseConfig` class provides three types of realtime subscriptions: (1) `subscribeToTable()` for PostgreSQL change notifications (insert/update/delete) with optional filters. (2) `subscribeToBroadcast()` for ephemeral non-persisted messages (e.g., typing indicators). (3) `subscribeToPresence()` for online/offline status tracking. The `CbtRealtimeService` uses these for exam session monitoring, attempt tracking, and anti-cheat event streaming. Channel cleanup is handled through `unsubscribeChannel()` and `unsubscribeAllChannels()` methods.

WARN No exponential backoff or reconnection strategy detected for realtime subscriptions. If the connection drops, the client may not gracefully recover. This is important for CBT exam sessions.

12. Storage

The `StorageService` class provides secure and non-secure persistence using `flutter_secure_storage` and `SharedPreferencesAsync`. Sensitive data (tokens, user ID, role) is encrypted at rest. The marketplace-download Edge Function generates signed URLs for the marketplace-products bucket with 1-hour expiry. However, no storage bucket policies or RLS policies for storage objects were found in the schema files. The health-check Edge Function does verify storage health by listing buckets.

WARN No storage bucket policies or object-level RLS policies found in SQL files. Must verify at runtime that marketplace-products and other buckets have proper access controls.

13. Environment Variables

The EnvConfig class loads environment variables from a .env asset file bundled with the Flutter app. In release builds, it falls back to compile-time --dart-define constants. In debug mode, safe placeholder values are used. The class has been properly hardened: server-only secrets (SUPABASE_SERVICE_KEY, FLUTTERWAVE_SECRET_KEY, FCM_SERVER_KEY, FLUTTERWAVE_WEBHOOK_SECRET_HASH) have been REMOVED from the client-side configuration. These keys must ONLY exist in Supabase Edge Function environment variables.

Variable	Location	Status
SUPABASE_URL	Client .env	MUST BE SET
SUPABASE_ANON_KEY	Client .env	MUST BE SET
FLUTTERWAVE_PUBLIC_KEY	Client .env	MUST BE SET
ENVIRONMENT	Client .env	MUST BE SET (production/staging/development)
SUPABASE_SERVICE_ROLE_KEY	Edge Function env	MUST BE SET (server-side only)
FLUTTERWAVE_SECRET_KEY	Edge Function env	RECEIVED - MUST BE SET
FLUTTERWAVE_WEBHOOK_SECRET_HASH	Edge Function env	MISSING - CRITICAL BLOCKER
OPENAI_API_KEY	Edge Function env	MUST BE SET (for AI features)
GEMINI_API_KEY	Edge Function env	MUST BE SET (for AI features)
DEPLOY_VERSION	Edge Function env	Optional
.env file	Client assets	DOES NOT EXIST - MUST BE CREATED

FAIL No .env file exists. All runtime configuration is missing. FLUTTERWAVE_WEBHOOK_SECRET_HASH is not provided. This blocks payment webhook verification.

14. Critical Blockers for Production

The following items are verified blockers that prevent production deployment. Each blocker is based on direct evidence from the source code review, not assumptions.

#	Blocker	Severity	Evidence
1	No .env file - all runtime configuration missing	CRITICAL	File does not exist; .env is listed as a required asset
2	Zero test coverage - no test/ directory	CRITICAL	No test files found anywhere in the project
3	FLUTTERWAVE_WEBHOOK_SECRET_HASH missing	CRITICAL	Required for webhook verification; not provided by user
4	RLS not verified on migration tables	HIGH	Only 8 base tables have RLS ENABLE; 23 migration tables
5	3 Edge Functions missing (create-plan, subscribe-plan, deactivate-plan)	HIGH	Direct code throws UnimplementedError for these operations
6	Local notification display not implemented	MEDIUM	TODO comment in notification_service.dart; flutter_local_notifications
7	No reconnection backoff for realtime subscriptions	MEDIUM	No exponential backoff strategy in CbtRealtimeService class
8	In-memory rate limiting for AI functions	MEDIUM	Resets on Edge Function cold start; not production-grade
9	751 TODO/FIXME markers across 134 files	MEDIUM	Indicates significant incomplete work, especially in extensions

#	Blocker	Severity	Evidence
10	No storage bucket policies verified	MEDIUM	No SQL policies found for marketplace-products or other

15. Verification Limitations

This Phase 1 audit was conducted through static source code analysis only. The following critical verifications could not be performed due to the absence of required tools on the verification system:

Verification	Required Tool	Status
flutter analyze	Flutter SDK	NOT INSTALLED
flutter test	Flutter SDK + test files	NOT INSTALLED + NO TESTS
flutter build web --release	Flutter SDK	NOT INSTALLED
Database schema query	Supabase CLI	NOT INSTALLED
RLS policy verification	Supabase CLI + SQL queries	NOT INSTALLED
Edge Function deployment status	Supabase CLI	NOT INSTALLED
Storage bucket policies	Supabase CLI	NOT INSTALLED
Runtime API testing	Supabase client + network	NOT AVAILABLE
Performance benchmarking	Supabase CLI + Flutter SDK	NOT AVAILABLE

To complete Phases 2-8 of the enterprise verification audit, the following environment must be set up: (1) Flutter SDK 3.3+ installed and configured. (2) Supabase CLI installed and authenticated. (3) Access to the Supabase project dashboard for runtime verification. (4) The .env file created with all required variables. (5) The FLUTTERWAVE_WEBHOOK_SECRET_HASH obtained from the Flutterwave dashboard. (6) Network access to the Supabase instance and Edge Functions.

16. Flutterwave Secret Key - Action Required

The Flutterwave Secret Key has been received and must be configured as a Supabase Edge Function environment variable. This key should NEVER be placed in the client-side .env file or in the Flutter application code. The correct configuration procedure is as follows:

Step 1: Set Edge Function Environment Variable

```
supabase secrets set  
FLUTTERWAVE_SECRET_KEY=FLWSECK-0725813e27cb7dae3faf8ce00ee35e4c-19fae2b082avt-X
```

Step 2: Obtain WEBHOOK_SECRET_HASH

Log into the Flutterwave dashboard, navigate to Settings > API Keys, and copy the Webhook Secret Hash. Then set it as another Edge Function environment variable:

```
supabase secrets set FLUTTERWAVE_WEBHOOK_SECRET_HASH=<your-webhook-secret-hash>
```

Step 3: Create .env File

Create a .env file in the project root with the following variables (client-side only, no secrets):

Variable	Example Value
SUPABASE_URL	https://your-project.supabase.co
SUPABASE_ANON_KEY	eyJhbGciOiJIUzI1NiIs...
FLUTTERWAVE_PUBLIC_KEY	FLWPUBK-xxxxxxxxxxxx
ENVIRONMENT	production

17. Next Steps for Phase 2

Based on the Phase 1 findings, the following actions are required before proceeding to Phase 2 (Resolve Blockers). These actions must be performed in a properly configured development environment.

Priority	Action	Owner
P0	Install Flutter SDK and Supabase CLI on verification system	DevOps
P0	Create .env file with all required client-side variables	Developer
P0	Set FLUTTERWAVE_SECRET_KEY as Supabase Edge Function env var	Developer
P0	Obtain and set FLUTTERWAVE_WEBHOOK_SECRET_HASH from Flutterwave dashboard	Developer
P0	Create test/ directory and write unit tests for critical services	Developer
P1	Run flutter analyze and fix all errors/warnings	Developer
P1	Verify RLS is enabled on all migration tables (not just base 8)	Database Admin
P1	Implement 3 missing Edge Functions (create-plan, subscribe-plan, transaction-plan)	Developer
P1	Implement local notification display with flutter_local_notifications	Developer
P2	Add reconnection backoff strategy for realtime subscriptions	Developer
P2	Implement persistent rate limiting for AI Edge Functions	Developer
P2	Define and enforce storage bucket policies	Database Admin
P2	Resolve TODO/FIXME markers (prioritize exam_ecosystem module)	Developer